SUPPLY MANAGEMENT SYSTEM — Functional Specification Document

FSD Addendum 47 — General Ledger Journal Engine

Property	Value
Document ID	SMS-FSD-ADD-047
Version	1.0
Date	2026-10-24
Status	Draft — Pending Approval
Author	System Architect
Classification	Internal — Development
Supersedes	None — extends `finance` schema established in A42
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18
Change Items	11 new features
Impact	8 migrations, 5 new entities, 2 modified entities, 8 new API endpoints, 4 new UI panels
Depends On	Addendum 42 (Chart of Accounts — FiscalYears, FiscalPeriods, ChartOfAccounts, AccountingDefaults), Addendum 44 (Financial Posting Gateway — IFinancialPostingGateway, FinanceDbContext, TenantFinanceConfigs)
Deferred	AR/AP Sub-Ledgers → A48, Bank Reconciliation → A49, Multi-Currency Engine → A50, Tax Configuration → A51, Financial Statements → A52
Naming Convention	Finance entities use standard accounting terminology. All entity names are singular. Field names use snake_case.

Table of Contents

1. Purpose & Scope
2. Background & Context
3. Design Principles
4. C1 — Journal Type Configuration
5. C2 — Fiscal Period Status Enhancement
6. C3 — Journal Entry Header
7. C4 — Journal Entry Lines
8. C5 — Entry Number Sequence Generation

9. C6 — Journal Entry Posting Engine
10. C7 — Journal Entry Reversal
11. C8 — Line-Level Reconciliation
12. C9 — Trial Balance Computation
13. C10 — Period Close & Year-End Close
14. C11 — Gateway Integration Enhancement
15. Database Migrations
16. API Endpoints
17. UI Wireframes & Specifications
18. Business Rules
19. Test Scenarios
20. Development Phases
21. Appendix A — Deferred Financial Addendum Roadmap

---

1. Purpose & Scope

This addendum implements the General Ledger Journal Engine — the accounting engine that sits behind the Financial Posting Gateway (A44) and operates on the Chart of Accounts foundation (A42). It is the first operational component of SMS's separate financial module.

1.1 Architectural Position

The GL Engine is **not** a new middleware layer. The middleware already exists — A44's `IFinancialPostingGateway`. This addendum provides the **implementation** behind the Internal GL Target that the gateway routes to:

Key constraint: The gateway contract (`IFinancialPostingGateway.PostAsync()`) does not change. Core modules continue making the same single call. The GL Engine enriches what happens inside Step 5.

1.2 Separation Guarantee

The GL Engine operates entirely within the `finance` schema and the `FinanceDbContext`:

- **No cross-schema foreign keys.** `JournalEntryLines.account_id` references `finance.ChartOfAccounts.id` (same schema). `JournalEntryLines.partner_id` is a plain `int` — no FK to `sales.Customers` or `purchase.Vendors`.
- **No cross-schema joins.** Trial balance, reconciliation, and period close operate exclusively on `finance.*` tables.

- **Source document reference is a soft link.** `JournalEntries.source_document_id` + `source_document_type` + `source_module` — three plain columns, no navigation properties.
- **Own DbContext.** All entities are registered in `FinanceDbContext` (A44). No entity touches `AppDbContext`.

When the financial system is extracted to a separate service, the DI registration for `IFinancialPostingGateway` swaps from the in-process implementation to an HTTP client. The GL Engine moves with the finance module. Core modules continue unchanged.

1.3 What This Addendum Does NOT Cover

Deferred Item	Reason	Target Addendum
AR/AP Sub-Ledger	Requires GL Engine as foundation; customer/vendor aging, statement, dunning	A48
Bank Reconciliation	Requires GL Engine for bank account JEs; match imported bank feeds to entries	A49
Multi-Currency Engine	Requires GL Engine for realized/unrealized gain/loss postings	A50
Tax Configuration & Reporting	Requires GL Engine for tax liability JEs; region-specific tax returns	A51
Financial Statements	Requires GL Engine + period close for Balance Sheet, P&L, Cash Flow	A52
Cost Center & Project Accounting	Dimensional analysis on journal lines	Future
Inter-Company Transactions	Multi-org JEs with elimination entries	Future
Budgeting & Variance Analysis	Budget vs. actual on GL accounts	Future

1.4 Scope Summary

Change ID	Title	Type	Impact
C1	Journal Type Configuration	New Entity + UI	1 new table (JournalTypes), seed data, 1 UI panel
C2	Fiscal Period Status Enhancement	Modified Entity	Add `AdjustmentOnly` status to FiscalPeriods (A42)
C3	Journal Entry Header	New Entity	1 new table (JournalEntries)
C4	Journal Entry Lines	New Entity	1 new table (JournalEntryLines)
C5	Entry Number Sequence Generation	New Entity + Service	1 new table (JournalEntrySequences), sequence service
C6	Journal Entry Posting Engine	Service	IJournalEntryService — create, validate, post
C7	Journal Entry Reversal	Service	Reverse a posted entry, generate counter-entry
C8	Line-Level Reconciliation	New Entity + Service	1 new table (ReconcileGroups), IReconciliationService
C9	Trial Balance Computation	Service + UI	ITrialBalanceService, trial balance UI panel
C10	Period Close & Year-End Close	Service + UI	IPeriodCloseService, close wizard UI panel
C11	Gateway Integration Enhancement	Modified Service	Enhance A44 JournalEntryBuilder to use GL Engine services

---

2. Background & Context

2.1 Current State (Post-A42 + A44)

Addendum 42 established the financial **foundation**:

- Chart of Accounts (4-level hierarchy, template-driven)
- Fiscal Years and Fiscal Periods (Open / Closed / Locked)
- Accounting Defaults (16 default account keys)
- Opening Balance entry mechanism

Addendum 44 established the financial **middleware**:

- `IFinancialPostingGateway` — single entry point for all modules
- `FinancialTransaction` — canonical DTO in SMS.Core
- `FinanceDbContext` — separate DbContext, separation-ready
- QuickBooks integration via outbox + Hangfire
- Configurable routing (internal GL, QB, both in parallel)

What is missing: The actual GL Engine. Today, the gateway's Step 5 creates a basic `JournalEntry` entity through `IJournalEntryBuilder`, but this builder does not:

- Validate the posting period is open
- Generate gap-free sequence numbers per journal type
- Enforce double-entry at the database level
- Update the Chart of Accounts running balances
- Support reversal with audit trail
- Provide reconciliation for AR/AP matching
- Compute trial balances
- Execute period close or year-end close

This addendum fills that gap.

2.2 Design Heritage

The GL Engine design follows the principle established during our Odoo/SAP comparison: **Odoo simplicity + SAP rigor**.

Feature	Source	SMS Adaptation
Two-table design (header + lines)	Odoo `account.move` / `account.move.line`	`JournalEntries` + `JournalEntryLines`
Journal Types with controlled numbering	SAP document types + number ranges	`JournalTypes` with per-type sequence prefix
Period status with AdjustmentOnly	SAP OB52 special periods	Add `AdjustmentOnly` to A42's FiscalPeriods
Reconciliation at line level	Odoo `account.full.reconcile` / `account.partial.reconcile`	`ReconcileGroups` with full/partial type
Multi-currency per line	SAP ACDOCA (4 currencies per line)	Transaction amount + local amount + exchange rate per line
Year-end close as single entry	Odoo year-end closing entry	Auto-generated closing JE (P&L → Retained Earnings)
No separate balance table	Both Odoo and SAP	Trial balance computed from JournalEntryLines on-the-fly
Idempotency key	A44 existing pattern	Carried forward on every JournalEntry

3. Design Principles

- **Double-Entry is Sacred** — Every journal entry must balance: total debits = total credits (within a tolerance of 0.01 for rounding). This is enforced at three levels: application validation in `IJournalEntryService`, database CHECK constraint on `JournalEntries`, and a database trigger that re-validates on UPDATE.
- **Period Gate Before Write** — No journal entry can be created or posted unless the target fiscal period's status is `Open` (or `AdjustmentOnly` for closing entries). This is checked in `IJournalEntryService.CreateAsync()` before any entity is added to the DbContext.
- **Sequence Numbers Are Gap-Free Per Type** — Each journal type (SALE, PURCH, CASH, BANK, MISC, OPEN, CLOSE) maintains its own sequence. Entry numbers follow the pattern `{prefix}/{fiscal_year}/{sequence}` (e.g., `SALE/2026/0001`). Gaps indicate deleted drafts and are logged but not backfilled.
- **Posted Entries Are Immutable** — Once a journal entry's status changes to `Posted`, its header and lines cannot be modified. The only way to correct a posted entry is through Reversal (C7), which creates a new counter-entry. Draft entries can be edited freely.
- **Reconciliation Is Line-Level** — Individual journal entry lines are reconciled, not entire entries. This supports partial payments (Customer pays 3 of 5 invoices), advance payments, and cross-entry matching. Reconciliation groups track full vs. partial reconciliation.
- **Trial Balance Is Computed, Not Stored** — No separate balance table. Trial balance is computed on-the-fly by aggregating `debit` and `credit` from `JournalEntryLines` where the parent `JournalEntries.status = 1 (Posted)`. The `ChartOfAccounts.current_balance` field (A42) is updated incrementally on posting for fast single-account lookups, but trial balance reports always recompute from lines for accuracy.
- **Year-End Close Creates One Entry** — Closing a fiscal year generates a single auto-entry that debits all Revenue accounts and credits all Expense accounts, posting the net P&L to the Retained Earnings account (from `AccountingDefaults.DEFAULT_RETAINED_EARNINGS`). Opening balances for the new year are the closing balances of the old year's Balance Sheet accounts (Assets, Liabilities, Equity).
- **All Within FinanceDbContext** — Every entity in this addendum is registered in `FinanceDbContext`. No references to `AppDbContext`. No cross-schema FKs.

---

4. C1 — Journal Type Configuration

4.1 Purpose

Journal types classify entries by their origin and control sequence numbering, default accounts, and posting behavior. They are the SMS equivalent of SAP document types and Odoo journal types.

4.2 Entity Definition — `finance.JournalTypes`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
org_id	int	NO	—	Multi-tenant discriminator
code	nvarchar(10)	NO	—	Short code (e.g., `SALE`, `PURCH`, `CASH`, `BANK`, `MISC`, `OPEN`, `CLOSE`)
name	nvarchar(100)	NO	—	Display name (e.g., "Sales Journal", "Purchase Journal")
type	tinyint	NO	—	0=Sale, 1=Purchase, 2=Cash, 3=Bank, 4=Miscellaneous, 5=Opening, 6=Closing
sequence_prefix	nvarchar(10)	NO	—	Prefix for entry numbers (e.g., `SALE`, `PURCH`, `JV`)
default_debit_account_id	int	YES	NULL	FK → finance.ChartOfAccounts.id. Default debit account for entries of this type
default_credit_account_id	int	YES	NULL	FK → finance.ChartOfAccounts.id. Default credit account for entries of this type
requires_partner	bit	NO	0	If true, every line in entries of this type must have a partner_id
auto_post	bit	NO	0	If true, entries of this type are auto-posted on creation (no Draft state). Used for system-generated entries (opening, closing)
is_system	bit	NO	0	If true, this type was seed-created and cannot be deleted (can be deactivated)
is_active	bit	NO	1	Soft-delete. Inactive types cannot be used for new entries
created_at	datetimeoffset	NO	SYSDATETIMEOFFSET()	

Column	Type	Nullable	Default	Description
created_by	nvarchar(450)	NO	—	
modified_at	datetimeoffset	YES	NULL	
modified_by	nvarchar(450)	YES	NULL	

Indexes:

Index	Columns	Type
UQ_JournalTypes_OrgCode	org_id, code	Unique
IX_JournalTypes_OrgType	org_id, type	Non-unique
IX_JournalTypes_Active	org_id, is_active	Non-unique, Filtered (WHERE is_active = 1)

Constraints:

- `code` must be uppercase alphanumeric, max 10 characters
- `type` CHECK: `type BETWEEN 0 AND 6`
- `default_debit_account_id` and `default_credit_account_id` must reference Level 4 (postable) accounts within the same `org_id`
- Only one journal type per `org_id` can have `type = 5` (Opening)
- Only one journal type per `org_id` can have `type = 6` (Closing)

4.3 Seed Data

When the Finance module is activated for a tenant (A42 CoA Setup), the system seeds 7 journal types:

code	name	type	sequence_prefix	requires_partner	auto_post	is_system
SALE	Sales Journal	0 (Sale)	SALE	true	false	true
PURCH	Purchase Journal	1 (Purchase)	PURCH	true	false	true
CASH	Cash Journal	2 (Cash)	CASH	false	false	true
BANK	Bank Journal	3 (Bank)	BANK	false	false	true
MISC	Miscellaneous Journal	4 (Misc)	JV	false	false	true
OPEN	Opening Journal	5 (Opening)	OPEN	false	true	true
CLOSE	Closing Journal	6 (Closing)	CLOSE	false	true	true

Tenants can create additional custom journal types but cannot delete system types — only deactivate them.

5. C2 — Fiscal Period Status Enhancement

5.1 Purpose

A42 defined three period statuses: `Open`, `Closed`, `Locked`. This addendum adds `AdjustmentOnly` — inspired by SAP's special periods — to support year-end adjustment entries after a period has been soft-closed to regular postings.

5.2 Modification to `finance.FiscalPeriods` (A42)

Changed column:

Column	Old CHECK	New CHECK
status	`IN ('Open', 'Closed', 'Locked')`	`IN ('Open', 'AdjustmentOnly', 'Closed', 'Locked')`

Status state machine (updated):

Transition rules:

From	To	Who Can Do It	Reversible
Open	AdjustmentOnly	Finance Admin	Yes → back to Open
AdjustmentOnly	Locked	Finance Admin	Yes → back to Open
Locked	Closed	Finance Admin	No (final)
Open	Locked	Finance Admin	Yes → back to Open
Locked	Open	Finance Admin (reopen)	—

Migration: `ALTER TABLE finance.FiscalPeriods DROP CONSTRAINT CK_status; ALTER TABLE finance.FiscalPeriods ADD CONSTRAINT CK_status CHECK (status IN ('Open', 'AdjustmentOnly',`

```
'Closed', 'Locked'));
```

---

6. C3 — Journal Entry Header

6.1 Purpose

The journal entry header records the metadata for each accounting transaction: what type it is, when it happened, which period it belongs to, its status, and its source document reference (soft link, no FK to other schemas).

6.2 Entity Definition — `finance.JournalEntries`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
org_id	int	NO	—	Multi-tenant discriminator
entry_number	nvarchar(30)	NO	—	Auto-generated: `{prefix}/{fiscal_year}/{sequence}` (e.g., `SALE/2026/0001`)
journal_type_id	int	NO	—	FK → finance.JournalTypes.id
entry_date	date	NO	—	Transaction date (determines fiscal period)
document_date	date	YES	NULL	Date on the source document (may differ from entry_date)
fiscal_year_id	int	NO	—	FK → finance.FiscalYears.id. Resolved from entry_date
fiscal_period_id	int	NO	—	FK → finance.FiscalPeriods.id. Resolved from entry_date
reference	nvarchar(100)	YES	NULL	Free-text reference (e.g., "INV-2026-0042", "Payment #789")
description	nvarchar(500)	YES	NULL	Narration / memo for the entire entry
source_document_type	nvarchar(30)	YES	NULL	`SaleInvoice`, `PurchaseInvoice`, `Payment`, `GoodsReceipt`, `ManualEntry`, etc.
source_document_id	int	YES	NULL	Plain int — no FK to any other schema. Soft link to source
source_module	nvarchar(20)	YES	NULL	`sales`, `purchase`, `pos`, `inventory`, `finance`
currency_id	int	NO	—	FK → lookups.Currencies (via code or id, depending on lookups design). Entry-level currency
exchange_rate	decimal(18,6)	YES	NULL	Exchange rate to org's reporting currency. NULL = 1.0 (same currency)
status	tinyint	NO	0	0 = Draft, 1 = Posted, 2 = Reversed

Column	Type	Nullable	Default	Description
posted_at	datetimeoffset	YES	NULL	Timestamp when status changed to Posted
posted_by	nvarchar(450)	YES	NULL	User who posted
reversed_by_entry_id	int	YES	NULL	FK → self. If this entry has been reversed, points to the reversal entry
reversal_of_entry_id	int	YES	NULL	FK → self. If this entry IS a reversal, points to the original entry
reversal_reason	nvarchar(200)	YES	NULL	Reason provided when reversing
idempotency_key	nvarchar(100)	NO	—	`{module}:{docType}:{docId}` or `manual:{guid}`. Prevents duplicate postings
is_opening_entry	bit	NO	0	true for opening balance entries
is_closing_entry	bit	NO	0	true for year-end closing entries
total_debit	decimal(18,4)	NO	0	Sum of all lines' debit amounts. Must equal total_credit
total_credit	decimal(18,4)	NO	0	Sum of all lines' credit amounts. Must equal total_debit
line_count	smallint	NO	0	Number of lines. Denormalized for display
created_at	datetimeoffset	NO	SYSDATETIMEOFFSET()	
created_by	nvarchar(450)	NO	—	
updated_at	datetimeoffset	YES	NULL	
updated_by	nvarchar(450)	YES	NULL	

Indexes:

Index	Columns	Type
UQ_JournalEntries_OrgId_EntryNumber	org_id, entry_number	Unique
UQ_JournalEntries_OrgId_IdempotencyKey	org_id, idempotency_key	Unique
IX_JournalEntries_OrgId_Status	org_id, status	Non-unique
IX_JournalEntries_OrgId_EntryDate	org_id, entry_date	Non-unique
IX_JournalEntries_JournalTypeId	journal_type_id	Non-unique
IX_JournalEntries_FiscalPeriodId	fiscal_period_id	Non-unique
IX_JournalEntries_FiscalYearId	fiscal_year_id	Non-unique
IX_JournalEntries_SourceDoc	org_id, source_document_type, source_document_id	Non-unique
IX_JournalEntries_PostedAt	org_id, posted_at	Non-unique, Filtered (WHERE status = 1)

Constraints:

- `status` CHECK: `status IN (0, 1, 2)`
- `CK_JE_Balanced` CHECK: `ABS(total_debit - total_credit) < 0.01`
- `entry_date` must fall within the date range of the referenced `fiscal_period_id`
- `fiscal_period_id` must belong to the referenced `fiscal_year_id`
- `reversed_by_entry_id` and `reversal_of_entry_id` cannot both be non-NULL on the same entry
- `journal_type_id` must be active (`is_active = 1`) at time of creation
- `is_opening_entry` and `is_closing_entry` cannot both be true

Foreign Keys (all within `finance` schema):

FK	References
journal_type_id	finance.JournalTypes.id
fiscal_year_id	finance.FiscalYears.id
fiscal_period_id	finance.FiscalPeriods.id
reversed_by_entry_id	finance.JournalEntries.id (self)
reversal_of_entry_id	finance.JournalEntries.id (self)

7. C4 — Journal Entry Lines

7.1 Purpose

Journal entry lines are the individual debit/credit postings against accounts. Each line references a ChartOfAccounts entry (Level 4 only) and optionally a partner (customer/vendor — plain int, no FK outside finance schema). Lines carry both transaction-currency and local-currency amounts for multi-currency readiness.

7.2 Entity Definition — `finance.JournalEntryLines`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
org_id	int	NO	—	Multi-tenant discriminator
journal_entry_id	int	NO	—	FK → finance.JournalEntries.id
line_number	smallint	NO	—	Sequential number within the entry (1, 2, 3…)
account_id	int	NO	—	FK → finance.ChartOfAccounts.id. Must be Level 4 (postable)
partner_id	int	YES	NULL	Plain int — Customer or Vendor ID. No FK to sales/purchase schemas
partner_type	nvarchar(10)	YES	NULL	`Customer`, `Vendor`, or NULL
description	nvarchar(500)	YES	NULL	Line-level narration
debit	decimal(18,4)	NO	0	Debit amount in entry currency
credit	decimal(18,4)	NO	0	Credit amount in entry currency
amount_currency	decimal(18,4)	YES	NULL	Amount in transaction currency (when different from entry currency). For multi-currency readiness (A50)
currency_id	int	YES	NULL	Transaction currency if different from entry header currency
exchange_rate	decimal(18,6)	YES	NULL	Line-level exchange rate override (NULL = use header rate)
local_debit	decimal(18,4)	NO	0	Debit amount in org's reporting currency. Computed: `debit * exchange_rate`
local_credit	decimal(18,4)	NO	0	Credit amount in org's reporting currency. Computed: `credit * exchange_rate`
cost_center_id	int	YES	NULL	Plain int — for future cost center/department analysis. No FK (reserved for Cost Center addendum)

Column	Type	Nullable	Default	Description
project_id	int	YES	NULL	Plain int — for future project accounting. No FK (reserved for Project Accounting addendum)
reconciled	bit	NO	0	true when this line has been fully reconciled
reconcile_group_id	int	YES	NULL	FK → finance.ReconcileGroups.id. Non-null when reconciled or partially reconciled
clearing_document_id	int	YES	NULL	FK → finance.JournalEntries.id. The entry that cleared this line
clearing_date	date	YES	NULL	Date when this line was cleared
amount_residual	decimal(18,4)	NO	0	Unreconciled amount. Initially = `debit - credit`. Decremented as reconciliation proceeds
tax_code_id	int	YES	NULL	Plain int — for future tax configuration. No FK (reserved for Tax addendum A51)
tax_amount	decimal(18,4)	YES	NULL	Tax amount included in or added to this line. For future Tax addendum
created_at	datetimeoffset	NO	SYSDATETIMEOFFSET()	

Indexes:

Index	Columns	Type
IX_JEL_Entry	journal_entry_id	Non-unique
IX_JEL_Account	org_id, account_id	Non-unique
IX_JEL_AccountClearing	org_id, account_id, clearing_date	Non-unique
IX_JEL_Partner	org_id, partner_id, partner_type, reconciled	Non-unique
IX_JEL_Reconcile	reconcile_group_id	Non-unique, Filtered (WHERE reconcile_group_id IS NOT NULL)
IX_JEL_OpenItems	org_id, account_id, reconciled	Non-unique, Filtered (WHERE reconciled = 0 AND (debit > 0 OR credit > 0))
IX_JEL_CostCenter	org_id, cost_center_id	Non-unique, Filtered (WHERE cost_center_id IS NOT NULL)

Constraints:

- `CK_JEL_DebitOrCredit` CHECK: `(debit > 0 AND credit = 0) OR (debit = 0 AND credit > 0) OR (debit = 0 AND credit = 0)` — a line is debit or credit, never both
- `line_number` must be unique within a `journal_entry_id`
- `account_id` must reference a Level 4, active account in `finance.ChartOfAccounts` with the same `org_id`
- When the parent entry's journal type has `requires_partner = true`, `partner_id` must not be NULL
- `partner_type` CHECK: `partner_type IN ('Customer', 'Vendor') OR partner_type IS NULL`
- `local_debit` and `local_credit` must be >= 0

Foreign Keys (all within `finance` schema):

FK	References
journal_entry_id	finance.JournalEntries.id (CASCADE DELETE)
account_id	finance.ChartOfAccounts.id
reconcile_group_id	finance.ReconcileGroups.id
clearing_document_id	finance.JournalEntries.id

8. C5 — Entry Number Sequence Generation

8.1 Purpose

Each journal type maintains its own gap-free sequence counter per fiscal year. Entry numbers follow the pattern `{sequence_prefix}/{fiscal_year_short}/{zero-padded-sequence}` (e.g., `SALE/2026/0001`, `PURCH/2026/0042`, `JV/2026/0003`).

8.2 Entity Definition — `finance.JournalEntrySequences`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
org_id	int	NO	—	Multi-tenant discriminator
journal_type_id	int	NO	—	FK → finance.JournalTypes.id
fiscal_year_id	int	NO	—	FK → finance.FiscalYears.id
current_sequence	int	NO	0	Last used sequence number
created_at	datetimeoffset	NO	SYSDATETIMEOFFSET()	
updated_at	datetimeoffset	YES	NULL	

Indexes:

Index	Columns	Type
UQ_JES_OrgTypeYear	org_id, journal_type_id, fiscal_year_id	Unique

Constraints:

- `current_sequence` must be >= 0
- Row-level locking (UPDLOCK, HOLDLOCK) during sequence increment to prevent duplicates under concurrency

8.3 Sequence Generation Logic

csharp

```
namespace SMS.Finance.Services;

public class JournalEntrySequenceService : IJournalEntrySequenceService
{
    private readonly FinanceDbContext _financeDb;

    public async Task<string> GenerateEntryNumberAsync(
        int orgId, int journalTypeId, int fiscalYearId)
    {
        // Row-level lock to prevent concurrent duplicate numbers
        var sequence = await _financeDb.JournalEntrySequences
            .FromSqlRaw(@"
                SELECT * FROM finance.JournalEntrySequences WITH (UPDLOCK, HOLDLOCK)
                WHERE org_id = {0} AND journal_type_id = {1} AND fiscal_year_id = {2}",
                orgId, journalTypeId, fiscalYearId)
            .FirstOrDefaultAsync();

        if (sequence == null)
        {
            // First entry of this type in this fiscal year — create sequence row
            sequence = new JournalEntrySequence
            {
                OrgId = orgId,
                JournalTypeId = journalTypeId,
                FiscalYearId = fiscalYearId,
                CurrentSequence = 0
            };
            _financeDb.JournalEntrySequences.Add(sequence);
        }

        sequence.CurrentSequence++;
        sequence.UpdatedAt = DateTimeOffset.UtcNow;

        // Fetch prefix and fiscal year for formatting
        var journalType = await _financeDb.JournalTypes
            .Where(jt => jt.Id == journalTypeId)
            .Select(jt => jt.SequencePrefix)
            .FirstAsync();

        var fiscalYear = await _financeDb.FiscalYears
            .Where(fy => fy.Id == fiscalYearId)
            .Select(fy => fy.StartDate.Year)
            .FirstAsync();

        // Format: SALE/2026/0001
        return $"{journalType}/{fiscalYear}/{sequence.CurrentSequence:D4}";
```

```
    }
}
```

Note: The sequence increment and the `JournalEntry` insert happen within the same `SaveChangesAsync()` call (same transaction), so a failed insert does not consume a sequence number. If the entry creation fails after sequence increment but before save, the transaction rolls back and the sequence is untouched.

---

9. C6 — Journal Entry Posting Engine

9.1 Purpose

The core service that creates, validates, and posts journal entries. This is the heart of the GL Engine.

9.2 Service Interface — `IJournalEntryService`

csharp

```
namespace SMS.Finance.Interfaces;

public interface IJournalEntryService
{
    /// <summary>
    /// Creates a journal entry in Draft status. Validates double-entry balance,
    /// period status, and account validity. Does NOT update CoA balances yet.
    /// </summary>
    Task<JournalEntryResult> CreateAsync(CreateJournalEntryRequest request);

    /// <summary>
    /// Transitions a Draft entry to Posted. Updates ChartOfAccounts.current_balance
    /// for each affected account. Sets posted_at and posted_by.
    /// </summary>
    Task<JournalEntryResult> PostAsync(int orgId, int journalEntryId, string postedBy);

    /// <summary>
    /// Creates and posts in one step (for system-generated entries from gateway).
    /// Equivalent to CreateAsync + PostAsync in a single transaction.
    /// </summary>
    Task<JournalEntryResult> CreateAndPostAsync(CreateJournalEntryRequest request);

    /// <summary>
    /// Updates a Draft entry. Posted entries cannot be modified (use ReverseAsync).
    /// </summary>
    Task<JournalEntryResult> UpdateDraftAsync(int orgId, int journalEntryId,
        UpdateJournalEntryRequest request);

    /// <summary>
    /// Deletes a Draft entry. Posted entries cannot be deleted (use ReverseAsync).
    /// </summary>
    Task DeleteDraftAsync(int orgId, int journalEntryId);

    /// <summary>
    /// Returns a single journal entry with all lines.
    /// </summary>
    Task<JournalEntryDetailDto> GetByIdAsync(int orgId, int journalEntryId);

    /// <summary>
    /// Paginated list with filters.
    /// </summary>
    Task<PagedResult<JournalEntryListDto>> ListAsync(int orgId, JournalEntryFilter filter);
}
```

9.3 Request / Response DTOs

csharp

```
public class CreateJournalEntryRequest
{
    public int OrgId { get; set; }
    public int JournalTypeId { get; set; }
    public DateTime EntryDate { get; set; }
    public DateTime? DocumentDate { get; set; }
    public string? Reference { get; set; }
    public string? Description { get; set; }
    public string? SourceDocumentType { get; set; }
    public int? SourceDocumentId { get; set; }
    public string? SourceModule { get; set; }
    public int CurrencyId { get; set; }
    public decimal? ExchangeRate { get; set; }
    public string IdempotencyKey { get; set; } = null!;
    public bool IsOpeningEntry { get; set; }
    public bool IsClosingEntry { get; set; }
    public List<CreateJournalEntryLineRequest> Lines { get; set; } = new();
    public string CreatedBy { get; set; } = null!;
}

public class CreateJournalEntryLineRequest
{
    public int AccountId { get; set; }
    public int? PartnerId { get; set; }
    public string? PartnerType { get; set; }
    public string? Description { get; set; }
    public decimal Debit { get; set; }
    public decimal Credit { get; set; }
    public decimal? AmountCurrency { get; set; }
    public int? CurrencyId { get; set; }
    public decimal? ExchangeRate { get; set; }
    public int? CostCenterId { get; set; }
    public int? ProjectId { get; set; }
    public int? TaxCodeId { get; set; }
    public decimal? TaxAmount { get; set; }
}

public class JournalEntryResult
{
    public bool Success { get; set; }
    public int? JournalEntryId { get; set; }
    public string? EntryNumber { get; set; }
    public string? ErrorCode { get; set; }
    public string? ErrorMessage { get; set; }
    public List<string> ValidationErrors { get; set; } = new();
}
```

```
public class JournalEntryFilter
{
    public int? JournalTypeId { get; set; }
    public int? FiscalYearId { get; set; }
    public int? FiscalPeriodId { get; set; }
    public byte? Status { get; set; }
    public DateTime? FromDate { get; set; }
    public DateTime? ToDate { get; set; }
    public int? AccountId { get; set; }
    public int? PartnerId { get; set; }
    public string? SourceModule { get; set; }
    public string? SearchText { get; set; }
    public int Page { get; set; } = 1;
    public int PageSize { get; set; } = 25;
    public string SortBy { get; set; } = "entry_date";
    public string SortDirection { get; set; } = "desc";
}
```

9.4 Validation Logic (CreateAsync)

The following validations run before creating a journal entry:

```
1. IDEMPOTENCY CHECK
   → SELECT COUNT(*) FROM finance.JournalEntries
     WHERE org_id = @OrgId AND idempotency_key = @Key
   → If exists, return existing entry (no duplicate)

2. JOURNAL TYPE VALIDATION
   → JournalType must exist, belong to this org, and be active
   → If requires_partner = true, every line must have partner_id

3. FISCAL PERIOD RESOLUTION
   → Find FiscalPeriod WHERE org_id = @OrgId
     AND start_date <= @EntryDate AND end_date >= @EntryDate
   → Period must exist (error: "No fiscal period covers this date")
   → Period status must be 'Open' — OR —
     Period status = 'AdjustmentOnly' AND (is_opening_entry OR is_closing_entry)
   → If status = 'Locked' or 'Closed' → error: "Period is locked/closed"

4. DOUBLE-ENTRY VALIDATION
   → SUM(lines.Debit) must equal SUM(lines.Credit) within 0.01
   → At least 2 lines required
   → Each line: debit > 0 XOR credit > 0 (not both, not neither for posted lines)

5. ACCOUNT VALIDATION (per line)
   → Account must exist in finance.ChartOfAccounts with same org_id
   → Account must be Level 4 (is_group = false)
   → Account must be active (is_active = true)
   → Account root_type must be appropriate for the debit/credit direction

6. CURRENCY VALIDATION
   → Currency must exist in lookups.Currencies
   → If entry currency differs from org's reporting currency,
     exchange_rate must be provided (> 0)

7. LOCAL AMOUNT COMPUTATION
   → For each line:
     local_debit = debit * (line.ExchangeRate ?? entry.ExchangeRate ?? 1.0)
     local_credit = credit * (line.ExchangeRate ?? entry.ExchangeRate ?? 1.0)
     amount_residual = debit - credit (signed, for reconciliation)
```

9.5 Posting Logic (PostAsync)

When a Draft entry is posted:

```
1. STATUS CHECK
   → Entry must be in status = 0 (Draft)
   → Re-validate period is still Open or AdjustmentOnly

2. STATUS UPDATE
   → entry.Status = 1 (Posted)
   → entry.PostedAt = DateTimeOffset.UtcNow
   → entry.PostedBy = currentUser

3. COA BALANCE UPDATE (per line)
   → For each line in the entry:
     UPDATE finance.ChartOfAccounts
     SET current_balance = current_balance + (local_debit - local_credit),
         has_transactions = 1
     WHERE id = @AccountId AND org_id = @OrgId

4. DOMAIN EVENT
   → Raise JournalEntryPostedEvent {OrgId, EntryId, EntryNumber, TotalDebit}
```

---

10. C7 — Journal Entry Reversal

10.1 Purpose

Posted entries cannot be edited or deleted. The only correction mechanism is reversal: creating a mirror entry that cancels the original's debits and credits.

10.2 Service Method

csharp

```csharp
// Added to IJournalEntryService
Task<JournalEntryResult> ReverseAsync(int orgId, int journalEntryId,
    string reason, DateTime? reversalDate, string reversedBy);
```

10.3 Reversal Logic

```
1. ORIGINAL ENTRY VALIDATION
   → Entry must be in status = 1 (Posted)
   → Entry must not already be reversed (reversed_by_entry_id IS NULL)

2. REVERSAL DATE
   → If reversalDate is null, use original entry's entry_date
   → Reversal date's fiscal period must be Open or AdjustmentOnly

3. CREATE REVERSAL ENTRY
   → New JournalEntry with:
     - journal_type_id = same as original
     - entry_date = reversalDate ?? original.entry_date
     - description = "Reversal of {original.entry_number}: {reason}"
     - source_document_type = "Reversal"
     - source_document_id = original.id
     - source_module = "finance"
     - reversal_of_entry_id = original.id
     - idempotency_key = "reversal:{original.id}"

4. MIRROR LINES
   → For each line in original:
     - Swap debit ↔ credit (original's debit becomes reversal's credit)
     - Same account_id, partner_id, partner_type
     - local_debit ↔ local_credit also swapped

5. AUTO-POST REVERSAL
   → The reversal entry is created and posted in one step

6. MARK ORIGINAL
   → original.Status = 2 (Reversed)
   → original.reversed_by_entry_id = reversal.id

7. COA BALANCE UPDATE
   → Same logic as posting, but with swapped amounts (net zero)

8. RECONCILIATION
   → Auto-reconcile the original and reversal lines that share
     the same account_id (they net to zero)
```

---

11. C8 — Line-Level Reconciliation

11.1 Purpose

Reconciliation matches debit and credit lines across different journal entries for the same account and/or partner. This is essential for AR (matching invoice entries with payment entries) and AP (matching bill

entries with payment entries).

11.2 Entity Definition — `finance.ReconcileGroups`

Column	Type	Nullable	Default	Description
id	int	NO	IDENTITY	Primary key
org_id	int	NO	—	Multi-tenant discriminator
group_number	nvarchar(20)	NO	—	Auto-generated: `REC/{year}/{sequence}`
reconcile_type	tinyint	NO	—	0 = Full (all lines net to zero), 1 = Partial (residual remains)
amount	decimal(18,4)	NO	—	Reconciled amount
currency_id	int	YES	NULL	Currency of the reconciliation
reconciled_at	datetimeoffset	NO	SYSDATETIMEOFFSET()	
created_by	nvarchar(450)	NO	—	

Indexes:

Index	Columns	Type
IX_ReconcileGroups_OrgId	org_id	Non-unique
UQ_ReconcileGroups_GroupNumber	org_id, group_number	Unique

11.3 Service Interface — `IReconciliationService`

```csharp
```

```
namespace SMS.Finance.Interfaces;

public interface IReconciliationService
{
    /// <summary>
    /// Reconcile a set of journal entry lines. If they net to zero,
    /// creates a Full reconciliation. If a residual remains, creates
    /// a Partial reconciliation and adjusts amount_residual on each line.
    /// </summary>
    Task<ReconcileResult> ReconcileAsync(ReconcileRequest request);

    /// <summary>
    /// Undo a reconciliation. Restores amount_residual on all lines
    /// in the group. Deletes the ReconcileGroup.
    /// </summary>
    Task UnreconcileAsync(int orgId, int reconcileGroupId);

    /// <summary>
    /// List unreconciled (open) lines for a given account and/or partner.
    /// Used by the reconciliation UI to show what can be matched.
    /// </summary>
    Task<IList<OpenItemDto>> GetOpenItemsAsync(
        int orgId, int accountId, int? partnerId, string? partnerType);

    /// <summary>
    /// Suggest reconciliation matches for a set of lines.
    /// Finds lines on the opposite side (debit vs. credit) that would
    /// net to zero or minimize residual.
    /// </summary>
    Task<IList<ReconcileSuggestion>> SuggestMatchesAsync(
        int orgId, int accountId, IList<int> selectedLineIds);
}
```

11.4 Reconciliation Logic

```
RECONCILE REQUEST:
  - lineIds: list of JournalEntryLine IDs to reconcile together
  - All lines must: same org_id, same account_id, be posted (parent entry status=1)
  - At least 2 lines

PROCESS:
  1. Sum all amount_residual values across selected lines
  2. If |sum| < 0.01 → Full Reconciliation (reconcile_type = 0)
     - Set reconciled = true, amount_residual = 0 on all lines
     - Create ReconcileGroup with reconcile_type = 0
  3. If |sum| >= 0.01 → Partial Reconciliation (reconcile_type = 1)
     - Reduce amount_residual on each line proportionally
     - The line with the largest absolute residual absorbs any remainder
     - Create ReconcileGroup with reconcile_type = 1
     - Lines remain reconciled = false (partial, still open)
  4. Set reconcile_group_id on all affected lines
  5. Set clearing_document_id and clearing_date on lines fully cleared

UNRECONCILE:
  1. Find all lines with reconcile_group_id = @GroupId
  2. Restore amount_residual = debit - credit (original) minus
     sum of other reconciliation groups still applied
  3. Clear reconciled, reconcile_group_id, clearing_document_id, clearing_date
  4. Delete the ReconcileGroup
```

---

12. C9 — Trial Balance Computation

12.1 Purpose

The trial balance is the foundational financial report — a listing of all accounts with their debit and credit totals for a given period. It is computed on-the-fly from `JournalEntryLines`, not from a pre-computed balance table.

12.2 Service Interface — `ITrialBalanceService`

csharp

```csharp
namespace SMS.Finance.Interfaces;

public interface ITrialBalanceService
{
    /// <summary>
    /// Generate a trial balance for a date range. Aggregates all posted
    /// journal entry lines grouped by account.
    /// </summary>
    Task<TrialBalanceReport> GenerateAsync(TrialBalanceRequest request);

    /// <summary>
    /// Get the running balance for a single account as of a specific date.
    /// </summary>
    Task<AccountBalanceDto> GetAccountBalanceAsync(
        int orgId, int accountId, DateTime asOfDate);

    /// <summary>
    /// Get the account ledger — all posted lines for one account in date order.
    /// Used for drilling down from trial balance.
    /// </summary>
    Task<PagedResult<AccountLedgerLineDto>> GetAccountLedgerAsync(
        int orgId, int accountId, DateTime fromDate, DateTime toDate,
        int page = 1, int pageSize = 50);
}
```

12.3 Request / Response DTOs

csharp

```
public class TrialBalanceRequest
{
    public int OrgId { get; set; }
    public DateTime FromDate { get; set; }
    public DateTime ToDate { get; set; }
    public int? FiscalYearId { get; set; }  // Alternative: by fiscal year
    public bool IncludeOpeningBalances { get; set; } = true;
    public bool IncludeClosingEntries { get; set; } = false;
    public int? CostCenterId { get; set; }   // Optional filter
    public int? ProjectId { get; set; }       // Optional filter
}

public class TrialBalanceReport
{
    public DateTime FromDate { get; set; }
    public DateTime ToDate { get; set; }
    public DateTime GeneratedAt { get; set; }
    public List<TrialBalanceLineDto> Lines { get; set; } = new();
    public decimal TotalDebit { get; set; }
    public decimal TotalCredit { get; set; }
    public bool IsBalanced => Math.Abs(TotalDebit - TotalCredit) < 0.01m;
}

public class TrialBalanceLineDto
{
    public int AccountId { get; set; }
    public string AccountCode { get; set; } = null!;
    public string AccountName { get; set; } = null!;
    public string RootType { get; set; } = null!;     // Asset, Liability, etc.
    public string? AccountType { get; set; }            // Cash, Bank, Receivable, etc.
    public int Level { get; set; }
    public decimal OpeningDebit { get; set; }
    public decimal OpeningCredit { get; set; }
    public decimal PeriodDebit { get; set; }
    public decimal PeriodCredit { get; set; }
    public decimal ClosingDebit { get; set; }
    public decimal ClosingCredit { get; set; }
}

public class AccountBalanceDto
{
    public int AccountId { get; set; }
    public string AccountCode { get; set; } = null!;
    public string AccountName { get; set; } = null!;
    public DateTime AsOfDate { get; set; }
    public decimal Debit { get; set; }
```

```
    public decimal Credit { get; set; }
    public decimal Balance { get; set; }  // Debit - Credit (positive = debit balance)
}
```

12.4 Computation Query

sql

```sql
-- Trial Balance computation (on-the-fly from posted lines)
SELECT
    coa.id              AS account_id,
    coa.account_code,
    coa.account_name,
    coa.root_type,
    coa.account_type,
    coa.level,
    -- Opening balance (all posted entries before @FromDate)
    ISNULL(SUM(CASE WHEN je.entry_date < @FromDate THEN jel.local_debit END), 0)  AS opening_
    ISNULL(SUM(CASE WHEN je.entry_date < @FromDate THEN jel.local_credit END), 0) AS opening_
    -- Period movement
    ISNULL(SUM(CASE WHEN je.entry_date BETWEEN @FromDate AND @ToDate THEN jel.local_debit END
    ISNULL(SUM(CASE WHEN je.entry_date BETWEEN @FromDate AND @ToDate THEN jel.local_credit EN
    -- Closing balance (opening + period)
    ISNULL(SUM(jel.local_debit), 0)   AS closing_debit,
    ISNULL(SUM(jel.local_credit), 0)  AS closing_credit
FROM finance.ChartOfAccounts coa
LEFT JOIN finance.JournalEntryLines jel
    ON jel.account_id = coa.id AND jel.org_id = coa.org_id
LEFT JOIN finance.JournalEntries je
    ON je.id = jel.journal_entry_id AND je.org_id = coa.org_id
    AND je.status = 1    -- Posted only
    AND je.entry_date <= @ToDate
    AND (@IncludeClosingEntries = 1 OR je.is_closing_entry = 0)
WHERE coa.org_id = @OrgId
    AND coa.is_group = 0    -- Level 4 only
    AND coa.is_active = 1
GROUP BY coa.id, coa.account_code, coa.account_name,
         coa.root_type, coa.account_type, coa.level
HAVING ISNULL(SUM(jel.local_debit), 0) > 0
    OR ISNULL(SUM(jel.local_credit), 0) > 0
ORDER BY coa.account_code;
```

Performance note: For large tenants with many journal entries, this query benefits from the `IX_JEL_Account` index on `(org_id, account_id)` and the `IX_JournalEntries_OrgId_Status` index with `status = 1` filter. No separate balance table is needed — SQL Server handles aggregation efficiently with proper indexing. If performance becomes an issue for very large datasets (100K+ entries per period), a

materialized view or periodic snapshot can be introduced as a future optimization without changing the service contract.

---

13. C10 — Period Close & Year-End Close

13.1 Purpose

Period close prevents further postings to a completed period. Year-end close generates the closing journal entry that transfers P&L balances to Retained Earnings, preparing the books for the new fiscal year.

13.2 Service Interface — `IPeriodCloseService`

csharp

```
namespace SMS.Finance.Interfaces;

public interface IPeriodCloseService
{
    /// <summary>
    /// Soft-close a period (Open → AdjustmentOnly or Locked).
    /// Prevents regular postings but allows adjustment entries in AdjustmentOnly.
    /// </summary>
    Task<CloseResult> ClosePeriodAsync(int orgId, int fiscalPeriodId,
        string targetStatus, string closedBy);

    /// <summary>
    /// Reopen a Locked period back to Open.
    /// Cannot reopen a Closed (final) period.
    /// </summary>
    Task ReopenPeriodAsync(int orgId, int fiscalPeriodId, string reopenedBy);

    /// <summary>
    /// Final-close a period (Locked → Closed). Irreversible.
    /// </summary>
    Task<CloseResult> FinalClosePeriodAsync(int orgId, int fiscalPeriodId, string closedBy);

    /// <summary>
    /// Close an entire fiscal year:
    /// 1. Validate all periods in the year are Locked or Closed
    /// 2. Generate the closing journal entry (P&L → Retained Earnings)
    /// 3. Mark the fiscal year as closed (is_closed = true)
    /// 4. Final-close all periods (Locked → Closed)
    /// </summary>
    Task<YearEndCloseResult> CloseYearAsync(int orgId, int fiscalYearId,
        int retainedEarningsAccountId, string closedBy);

    /// <summary>
    /// Preview what the year-end closing entry would look like
    /// without actually creating it. Used by the UI to show the
    /// closing entry before the user confirms.
    /// </summary>
    Task<YearEndPreview> PreviewYearEndCloseAsync(int orgId, int fiscalYearId,
        int retainedEarningsAccountId);

    /// <summary>
    /// Get the close status of all periods in a fiscal year.
    /// </summary>
    Task<IList<PeriodCloseStatusDto>> GetPeriodStatusesAsync(
        int orgId, int fiscalYearId);
}
```

13.3 Request / Response DTOs

csharp

```
public class CloseResult
{
    public bool Success { get; set; }
    public string? ErrorMessage { get; set; }
    public int AffectedPeriodId { get; set; }
    public string NewStatus { get; set; } = null!;
}

public class YearEndCloseResult
{
    public bool Success { get; set; }
    public string? ErrorMessage { get; set; }
    public int? ClosingEntryId { get; set; }
    public string? ClosingEntryNumber { get; set; }
    public decimal TotalRevenueClosing { get; set; }
    public decimal TotalExpenseClosing { get; set; }
    public decimal NetIncomeTransferred { get; set; }  // Revenue - Expense → Retained Earnin
}

public class YearEndPreview
{
    public decimal TotalRevenue { get; set; }
    public decimal TotalExpenses { get; set; }
    public decimal NetIncome { get; set; }
    public int RetainedEarningsAccountId { get; set; }
    public string RetainedEarningsAccountName { get; set; } = null!;
    public List<YearEndPreviewLine> Lines { get; set; } = new();
}

public class YearEndPreviewLine
{
    public int AccountId { get; set; }
    public string AccountCode { get; set; } = null!;
    public string AccountName { get; set; } = null!;
    public string RootType { get; set; } = null!;
    public decimal Balance { get; set; }
    public decimal ClosingDebit { get; set; }   // Amount that will be debited to close
    public decimal ClosingCredit { get; set; }  // Amount that will be credited to close
}

public class PeriodCloseStatusDto
{
    public int PeriodId { get; set; }
    public string PeriodName { get; set; } = null!;
    public int PeriodNumber { get; set; }
    public string Status { get; set; } = null!;
```

```
    public int PostedEntryCount { get; set; }
    public int DraftEntryCount { get; set; }
    public bool HasUnpostedDrafts => DraftEntryCount > 0;
}
```

13.4 Year-End Closing Logic

```
YEAR-END CLOSE PROCESS:

1. PRE-CHECKS
   → All 12 periods (1–12) must be in status 'Locked' or 'Closed'
   → Period 13 (if exists) must be 'Open' or 'AdjustmentOnly'
   → fiscal_year.is_closed must be false
   → No Draft entries exist in any period of this fiscal year
   → retainedEarningsAccountId must be a valid Level 4 Equity account

2. COMPUTE P&L BALANCES
   → For all Revenue accounts (root_type = 'Revenue'):
     revenue_balance = SUM(local_credit) - SUM(local_debit) for this fiscal year
   → For all Expense accounts (root_type = 'Expense'):
     expense_balance = SUM(local_debit) - SUM(local_credit) for this fiscal year
   → Net Income = revenue_balance - expense_balance

3. GENERATE CLOSING JOURNAL ENTRY
   → Journal Type = CLOSE (type = 6)
   → Entry Date = last day of fiscal year (fiscal_year.end_date)
   → Fiscal Period = Period 12 (or 13 if exists)
   → is_closing_entry = true
   → Idempotency Key = "yearclose:{fiscal_year_id}"

   Lines:
   → For EACH Revenue account with a balance:
     - Debit the revenue account (zeroing its balance)
   → For EACH Expense account with a balance:
     - Credit the expense account (zeroing its balance)
   → One Retained Earnings line:
     - If Net Income > 0: Credit Retained Earnings (profit)
     - If Net Income < 0: Debit Retained Earnings (loss)
     - If Net Income = 0: No Retained Earnings line needed

4. POST THE CLOSING ENTRY
   → Auto-post (journal type CLOSE has auto_post = true)
   → Updates ChartOfAccounts.current_balance for all affected accounts

5. MARK FISCAL YEAR AS CLOSED
   → fiscal_year.is_closed = true

6. FINAL-CLOSE ALL PERIODS
   → All periods (1–13) → status = 'Closed'
```

```
7. DOMAIN EVENT
   → Raise FiscalYearClosedEvent {OrgId, FiscalYearId, ClosingEntryId, NetIncome}
```

---

14. C11 — Gateway Integration Enhancement

14.1 Purpose

Enhance the existing `IJournalEntryBuilder` (A44) to use the new GL Engine services. The gateway contract (`IFinancialPostingGateway.PostAsync()`) does not change — only the internal implementation of the builder that creates journal entries at Step 5.

14.2 Enhanced JournalEntryBuilder

The current A44 builder creates a basic `JournalEntry` entity. The enhanced builder delegates to `IJournalEntryService`:

csharp

```
namespace SMS.Finance.Gateway;

/// <summary>
/// Enhanced builder that uses the GL Engine (A47) instead of
/// directly constructing JournalEntry entities.
/// The gateway still calls _journalBuilder.BuildAsync() —
/// but now it goes through full GL Engine validation and sequencing.
/// </summary>
public class JournalEntryBuilder : IJournalEntryBuilder
{
    private readonly IJournalEntryService _journalEntryService;
    private readonly IJournalTypeResolver _journalTypeResolver;

    public async Task<JournalEntry> BuildAsync(FinancialTransaction transaction)
    {
        // 1. Resolve journal type from transaction.DocumentType
        var journalTypeId = await _journalTypeResolver.ResolveAsync(
            transaction.OrgId, transaction.DocumentType);

        // 2. Build the CreateJournalEntryRequest from the canonical DTO
        var request = new CreateJournalEntryRequest
        {
            OrgId = transaction.OrgId,
            JournalTypeId = journalTypeId,
            EntryDate = transaction.TransactionDate,
            DocumentDate = transaction.DocumentDate,
            Reference = transaction.SourceDocumentNumber,
            Description = transaction.Description,
            SourceDocumentType = transaction.DocumentType.ToString(),
            SourceDocumentId = transaction.SourceDocumentId,
            SourceModule = transaction.SourceModule,
            CurrencyId = transaction.CurrencyId,
            ExchangeRate = transaction.ExchangeRate,
            IdempotencyKey = transaction.IdempotencyKey,
            CreatedBy = transaction.CreatedBy,
            Lines = transaction.Lines.Select((l, i) => new CreateJournalEntryLineRequest
            {
                AccountId = l.AccountId,    // Resolved from AccountCode by gateway
                PartnerId = l.PartnerId,
                PartnerType = l.PartnerType,
                Description = l.Description,
                Debit = l.Debit,
                Credit = l.Credit,
                CurrencyId = l.CurrencyId,
                ExchangeRate = l.ExchangeRate
            }).ToList()
```

```
        };

        // 3. Delegate to GL Engine — full validation, sequencing, period check
        var result = await _journalEntryService.CreateAndPostAsync(request);

        if (!result.Success)
        {
            throw new JournalEntryPostingException(
                result.ErrorCode, result.ErrorMessage, result.ValidationErrors);
        }

        // 4. Return the created entity (already tracked by FinanceDbContext)
        return await _journalEntryService.GetEntityByIdAsync(
            transaction.OrgId, result.JournalEntryId!.Value);
    }
}
```

14.3 Journal Type Resolver

Maps `FinancialTransaction.DocumentType` to a `JournalTypes.id`:

DocumentType (A44 enum)	Journal Type Code	Resolution
SaleInvoice	SALE	type = 0 (Sale)
SaleCreditNote	SALE	type = 0 (Sale)
PurchaseInvoice	PURCH	type = 1 (Purchase)
PurchaseCreditNote	PURCH	type = 1 (Purchase)
Payment	CASH or BANK	Determined by payment method
Receipt	CASH or BANK	Determined by payment method
InventoryAdjustment	MISC	type = 4 (Misc)
ManualJournalEntry	MISC	type = 4 (Misc)
OpeningBalance	OPEN	type = 5 (Opening)
YearEndClosing	CLOSE	type = 6 (Closing)

15. Database Migrations

Migration ID	Name	Description	Change Item
M1	CreateJournalTypes	Create `finance.JournalTypes` table, seed 7 system types per org	C1
M2	AlterFiscalPeriodStatus	Add `AdjustmentOnly` to `finance.FiscalPeriods.status` CHECK constraint	C2
M3	CreateJournalEntries	Create `finance.JournalEntries` table with all indexes and constraints	C3
M4	CreateJournalEntryLines	Create `finance.JournalEntryLines` table with all indexes and constraints	C4
M5	CreateJournalEntrySequences	Create `finance.JournalEntrySequences` table	C5
M6	CreateReconcileGroups	Create `finance.ReconcileGroups` table	C8
M7	AddChartOfAccountsHasTransactions	Add `has_transactions` flag update trigger on JournalEntryLines insert	C6
M8	SeedJournalTypesForExistingTenants	For tenants who already have CoA setup complete, seed the 7 journal types	C1

Migration Dependencies:

```
M1 (JournalTypes)
  └── M3 depends on M1 (JournalEntries.journal_type_id → JournalTypes.id)
        └── M4 depends on M3 (JournalEntryLines.journal_entry_id → JournalEntries.id)
        └── M5 depends on M1, M3 (JournalEntrySequences references JournalTypes +
FiscalYears)
M2 (FiscalPeriod status) — independent
M6 (ReconcileGroups) — independent, but M4 references it
M7 (Trigger) — depends on M4
M8 (Seed data) — depends on M1
```

Execution order: M1 → M2 → M3 → M6 → M4 → M5 → M7 → M8

---

16. API Endpoints

16.1 Journal Types

Method	Route	Description	Auth
GET	`/api/v1/finance/journal-types`	List all journal types for the org	Finance.View
GET	`/api/v1/finance/journal-types/{id}`	Get a single journal type	Finance.View
POST	`/api/v1/finance/journal-types`	Create a custom journal type	Finance.Admin
PUT	`/api/v1/finance/journal-types/{id}`	Update a journal type	Finance.Admin
PATCH	`/api/v1/finance/journal-types/{id}/deactivate`	Deactivate a journal type	Finance.Admin

16.2 Journal Entries

Method	Route	Description	Auth
GET	`/api/v1/finance/journal-entries`	List journal entries (paginated, filtered)	Finance.View
GET	`/api/v1/finance/journal-entries/{id}`	Get entry with all lines	Finance.View
POST	`/api/v1/finance/journal-entries`	Create a manual journal entry (Draft)	Finance.JournalEntry.Create
PUT	`/api/v1/finance/journal-entries/{id}`	Update a Draft entry	Finance.JournalEntry.Edit
DELETE	`/api/v1/finance/journal-entries/{id}`	Delete a Draft entry	Finance.JournalEntry.Delete
POST	`/api/v1/finance/journal-entries/{id}/post`	Post a Draft entry	Finance.JournalEntry.Post
POST	`/api/v1/finance/journal-entries/{id}/reverse`	Reverse a Posted entry	Finance.JournalEntry.Reverse

16.3 Reconciliation

Method	Route	Description	Auth
GET	`/api/v1/finance/reconciliation/open-items`	List unreconciled lines (by account, partner)	Finance.Reconcile.View
POST	`/api/v1/finance/reconciliation/suggest`	Get reconciliation suggestions	Finance.Reconcile.View
POST	`/api/v1/finance/reconciliation/reconcile`	Reconcile a set of lines	Finance.Reconcile.Execute
POST	`/api/v1/finance/reconciliation/{groupId}/unreconcile`	Undo a reconciliation	Finance.Reconcile.Execute

16.4 Trial Balance & Reports

Method	Route	Description	Auth
GET	`/api/v1/finance/trial-balance`	Generate trial balance for date range	Finance.Reports.View
GET	`/api/v1/finance/accounts/{id}/balance`	Get account balance as of date	Finance.Reports.View
GET	`/api/v1/finance/accounts/{id}/ledger`	Get account ledger (line-by-line)	Finance.Reports.View

16.5 Period Close

Method	Route	Description	Auth
GET	`/api/v1/finance/period-close/{fiscalYearId}/status`	Get period close status for all periods in a year	Finance.PeriodClose.View
POST	`/api/v1/finance/period-close/{periodId}/close`	Close or lock a period	Finance.PeriodClose.Execute
POST	`/api/v1/finance/period-close/{periodId}/reopen`	Reopen a locked period	Finance.PeriodClose.Execute
POST	`/api/v1/finance/period-close/{periodId}/final-close`	Final close (irreversible)	Finance.PeriodClose.Admin
GET	`/api/v1/finance/year-end-close/{fiscalYearId}/preview`	Preview year-end closing entry	Finance.PeriodClose.View
POST	`/api/v1/finance/year-end-close/{fiscalYearId}/execute`	Execute year-end close	Finance.PeriodClose.Admin

---

17. UI Wireframes & Specifications

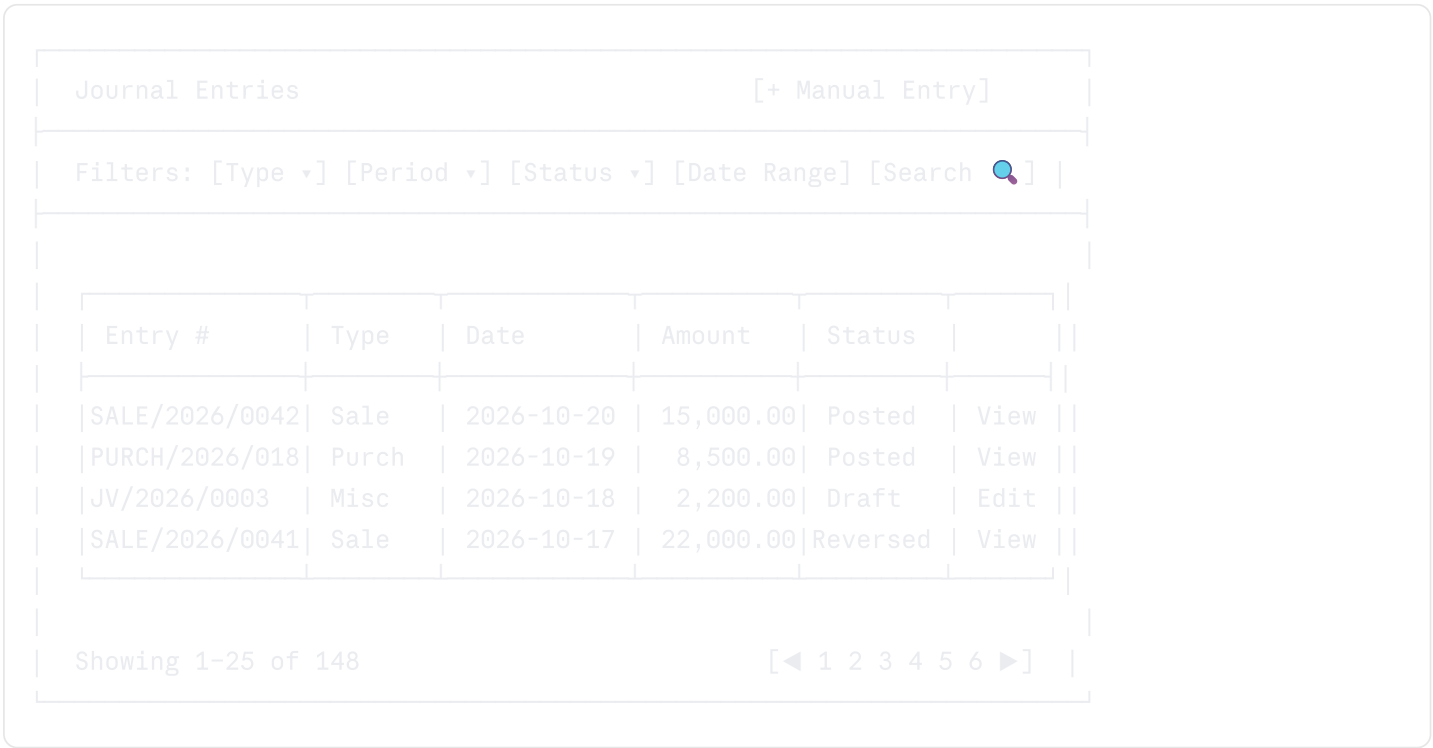
17.1 Journal Type Management Panel

Route: `/finance/settings/journal-types`



17.2 Journal Entry List Panel

Route: `/finance/journal-entries`



17.3 Trial Balance Panel

Route: `/finance/reports/trial-balance`

17.4 Period Close Panel

Route: `/finance/period-close`

Period Close — FY 2026
[Fiscal Year ▾]
| Period | Entries | Drafts | Status | Action
1 | January 2026 | 42 | 0 | Closed | —
2 | February 2026 | 38 | 0 | Closed | —
... | ... | ... | ... | ... | ...
9 | September | 51 | 0 | Locked | [Reopen] [Close]
10 | October 2026 | 34 | 3 | Open | [Lock]
11 | November 2026 | 0 | 0 | Open | [Lock]
12 | December 2026 | 0 | 0 | Open | [Lock]
▲ October has 3 unposted drafts. Post or delete them before locking the period.
Year-End Close
All periods must be Locked or Closed before year-end close
Retained Earnings Account: [3201 — Retained Earnings ▾]
[Preview Closing Entry]
[Execute Year-End Close]

18. Business Rules

18.1 Journal Entry Rules

Rule ID	Description	Enforcement
BR-JE-001	Every journal entry must have total_debit = total_credit (within 0.01 tolerance)	Application + CHECK constraint
BR-JE-002	A journal entry must have at least 2 lines	Application validation
BR-JE-003	Each line must be debit OR credit, never both (debit > 0 XOR credit > 0)	Application + CHECK constraint
BR-JE-004	Entry date must fall within an Open or AdjustmentOnly fiscal period	Application validation in CreateAsync
BR-JE-005	AdjustmentOnly periods accept only entries where is_opening_entry = true OR is_closing_entry = true	Application validation
BR-JE-006	Each line's account_id must reference a Level 4, active ChartOfAccounts entry	Application validation
BR-JE-007	Idempotency key must be unique per org. Duplicate submissions return the existing entry	Application + UNIQUE constraint
BR-JE-008	Posted entries (status = 1) cannot be modified or deleted	Application enforcement
BR-JE-009	Only Draft entries (status = 0) can be posted	Application validation in PostAsync
BR-JE-010	Entry number is auto-generated and immutable after creation	Application enforcement
BR-JE-011	When journal type has requires_partner = true, every line must have partner_id	Application validation
BR-JE-012	When entry currency differs from org reporting currency, exchange_rate must be > 0	Application validation
BR-JE-013	local_debit and local_credit are computed as debit * rate and credit * rate respectively	Application computation

18.2 Reversal Rules

Rule ID	Description	Enforcement
BR-RV-001	Only Posted entries (status = 1) can be reversed	Application validation
BR-RV-002	An entry can be reversed only once (reversed_by_entry_id must be NULL)	Application validation
BR-RV-003	Reversal date's fiscal period must be Open or AdjustmentOnly	Application validation
BR-RV-004	Reversal creates a mirror entry with debits and credits swapped	Application logic
BR-RV-005	Reversal entry is auto-posted (no Draft state)	Application logic
BR-RV-006	Original entry's status changes to Reversed (2) with reversed_by_entry_id set	Application logic
BR-RV-007	Reversal entry's reversal_of_entry_id points to the original	Application logic
BR-RV-008	Original and reversal lines on the same account are auto-reconciled	Application logic

18.3 Reconciliation Rules

Rule ID	Description	Enforcement
BR-RC-001	Only lines from Posted entries (status = 1) can be reconciled	Application validation
BR-RC-002	All lines in a reconciliation must belong to the same account_id and org_id	Application validation
BR-RC-003	A minimum of 2 lines is required for reconciliation	Application validation
BR-RC-004	If selected lines net to zero (within 0.01), create Full reconciliation	Application logic
BR-RC-005	If selected lines have a residual, create Partial reconciliation	Application logic
BR-RC-006	Fully reconciled lines have reconciled = true, amount_residual = 0	Application logic
BR-RC-007	Unreconcile restores amount_residual and clears reconcile_group_id	Application logic
BR-RC-008	A line can belong to only one active ReconcileGroup at a time	Application + constraint

18.4 Period Close Rules

Rule ID	Description	Enforcement
BR-PC-001	A period can be locked only if it has no Draft entries	Application validation
BR-PC-002	A Locked period can be reopened (→ Open). A Closed period cannot	Application validation
BR-PC-003	Final close (Locked → Closed) is irreversible	Application enforcement
BR-PC-004	Year-end close requires all periods (1–12) to be Locked or Closed	Application validation
BR-PC-005	Year-end close generates one closing JE: Revenue Dr / Expense Cr / Retained Earnings	Application logic
BR-PC-006	Year-end close marks fiscal_year.is_closed = true	Application logic
BR-PC-007	No entries can be posted to a closed fiscal year	Application validation
BR-PC-008	Retained Earnings account must be a Level 4 Equity account from AccountingDefaults	Application validation
BR-PC-009	Year-end closing entry's idempotency key = "yearclose:{fiscal_year_id}" — prevents duplicate close	Application + UNIQUE constraint

18.5 Trial Balance Rules

Rule ID	Description	Enforcement
BR-TB-001	Trial balance includes only Posted entries (status = 1)	Query filter
BR-TB-002	Trial balance total debit must equal total credit	Validation (assertion, not enforcement)
BR-TB-003	Opening balances = sum of all posted lines before the from_date	Query logic
BR-TB-004	Closing entries are excluded unless IncludeClosingEntries = true	Query filter
BR-TB-005	Accounts with zero activity are excluded from the report	HAVING clause

19. Test Scenarios

19.1 Journal Entry Creation

ID	Scenario	Expected Result
TS-JE-001	Create a balanced journal entry with 2 lines (debit 1000, credit 1000) in an Open period	Entry created in Draft, entry_number generated, total_debit = total_credit = 1000
TS-JE-002	Create an unbalanced entry (debit 1000, credit 999)	Rejected: "Entry is not balanced"
TS-JE-003	Create an entry with both debit and credit > 0 on same line	Rejected: "Line cannot have both debit and credit"
TS-JE-004	Create an entry with entry_date in a Closed period	Rejected: "Period is closed"
TS-JE-005	Create an entry with entry_date in a Locked period	Rejected: "Period is locked"
TS-JE-006	Create an entry with entry_date in an AdjustmentOnly period (not opening/closing)	Rejected: "Period accepts only adjustment entries"
TS-JE-007	Create a closing entry in an AdjustmentOnly period	Entry created successfully
TS-JE-008	Create an entry with the same idempotency_key as an existing entry	Returns existing entry (no duplicate)
TS-JE-009	Create an entry referencing a Level 3 (group) account	Rejected: "Account is not postable"
TS-JE-010	Create an entry referencing an inactive account	Rejected: "Account is inactive"
TS-JE-011	Create a SALE type entry without partner_id on a line	Rejected: "Sales entries require a partner"
TS-JE-012	Create an entry with only 1 line	Rejected: "Minimum 2 lines required"
TS-JE-013	Create an entry with entry_date outside any fiscal period	Rejected: "No fiscal period covers this date"
TS-JE-014	Create entry in foreign currency without exchange_rate	Rejected: "Exchange rate required for foreign currency"
TS-JE-015	Create entry in foreign currency with exchange_rate = 3.67	local_debit = debit * 3.67, local_credit = credit * 3.67

19.2 Journal Entry Posting

ID	Scenario	Expected Result
TS-JE-016	Post a Draft entry	Status → Posted, posted_at set, CoA balances updated
TS-JE-017	Post an already-Posted entry	Rejected: "Entry is already posted"
TS-JE-018	Post a Reversed entry	Rejected: "Entry has been reversed"
TS-JE-019	Post a Draft entry whose period was Locked between creation and posting	Rejected: "Period is locked"
TS-JE-020	Post entry → verify ChartOfAccounts.current_balance updated	Debit account balance increased, credit account balance increased
TS-JE-021	Post entry → verify ChartOfAccounts.has_transactions = true	Both accounts' has_transactions set to true

19.3 Journal Entry Reversal

ID	Scenario	Expected Result
TS-RV-001	Reverse a Posted entry	Original → Reversed, new mirror entry created and Posted, CoA balances net zero
TS-RV-002	Reverse an already-Reversed entry	Rejected: "Entry has already been reversed"
TS-RV-003	Reverse a Draft entry	Rejected: "Only posted entries can be reversed"
TS-RV-004	Reverse with a reversal_date in a Closed period	Rejected: "Reversal period is closed"
TS-RV-005	Reverse entry → verify original and reversal lines are auto-reconciled	Lines with same account_id reconciled, amount_residual = 0
TS-RV-006	Reverse entry with reason "Incorrect amount"	reversal_reason stored, reversal entry description = "Reversal of SALE/2026/0042: Incorrect amount"

19.4 Sequence Generation

ID	Scenario	Expected Result
TS-SQ-001	First SALE entry in FY 2026	entry_number = "SALE/2026/0001"
TS-SQ-002	Second SALE entry in FY 2026	entry_number = "SALE/2026/0002"
TS-SQ-003	First PURCH entry in FY 2026 (SALE already at 0002)	entry_number = "PURCH/2026/0001" (independent sequence)
TS-SQ-004	Concurrent entry creation (2 requests simultaneously)	Both get unique numbers (row-level locking), no duplicates
TS-SQ-005	Delete a Draft entry, create new entry	New entry gets next sequence (no gap backfill)

19.5 Reconciliation

ID	Scenario	Expected Result
TS-RC-001	Reconcile invoice debit 1000 with payment credit 1000	Full reconciliation: both lines reconciled = true, amount_residual = 0
TS-RC-002	Reconcile invoice debit 1000 with partial payment credit 600	Partial reconciliation: invoice amount_residual = 400, payment amount_residual = 0
TS-RC-003	Reconcile 3 invoices (1000 + 500 + 300) with 1 payment (1800)	Full reconciliation across all 4 lines
TS-RC-004	Unreconcile a Full reconciliation	All lines: reconciled = false, amount_residual restored, ReconcileGroup deleted
TS-RC-005	Attempt to reconcile lines from different accounts	Rejected: "All lines must belong to the same account"
TS-RC-006	Attempt to reconcile a line from a Draft entry	Rejected: "Only lines from posted entries can be reconciled"
TS-RC-007	Suggest matches for an invoice line of 5000	Returns credit lines on same account where amount_residual sums near 5000

19.6 Trial Balance

ID	Scenario	Expected Result
TS-TB-001	Generate trial balance for October 2026	Debit total = Credit total, all accounts with activity listed
TS-TB-002	Generate trial balance with IncludeOpeningBalances = true	Opening column shows balances before from_date
TS-TB-003	Generate trial balance with IncludeClosingEntries = false	Year-end closing entries excluded
TS-TB-004	Get account balance for "Cash at Bank" as of 2026-10-15	Returns sum of all posted debits and credits through that date
TS-TB-005	Get account ledger for "Accounts Receivable" October 2026	Returns all posted lines with running balance
TS-TB-006	Draft entries should not appear in trial balance	Only status = 1 lines aggregated

19.7 Period Close & Year-End

ID	Scenario	Expected Result
TS-PC-001	Lock a period with no drafts	Status → Locked
TS-PC-002	Lock a period that has 2 Draft entries	Rejected: "Period has 2 unposted draft entries"
TS-PC-003	Reopen a Locked period	Status → Open
TS-PC-004	Reopen a Closed (final) period	Rejected: "Closed periods cannot be reopened"
TS-PC-005	Final-close a Locked period	Status → Closed (irreversible)
TS-PC-006	Execute year-end close with all periods Locked	Closing JE created: Revenue debited, Expense credited, Retained Earnings = net P&L
TS-PC-007	Execute year-end close with period 10 still Open	Rejected: "All periods must be Locked or Closed"
TS-PC-008	Execute year-end close twice	Second attempt returns existing closing entry (idempotency)
TS-PC-009	Preview year-end close	Returns preview with all Revenue/Expense accounts and net income calculation
TS-PC-010	Post an entry to a closed fiscal year	Rejected: "Fiscal year is closed"
TS-PC-011	Year-end close with net loss	Retained Earnings debited (loss transferred)

19.8 Gateway Integration

ID	Scenario	Expected Result
TS-GW-001	Sale invoice posted → gateway PostAsync → GL Engine creates SALE type JE	Entry created with correct journal type, period, sequence number
TS-GW-002	Purchase invoice posted → gateway PostAsync → GL Engine creates PURCH type JE	Entry created with PURCH prefix
TS-GW-003	Gateway PostAsync with internal_accounting_enabled = true, QB active	JE created (sync) + outbox entry (async) — parallel
TS-GW-004	Gateway PostAsync with internal_accounting_enabled = false, QB active	No JE created, only outbox entry
TS-GW-005	Gateway PostAsync when target period is Closed	JE creation fails, gateway returns error, outbox entry still written (QB gets it regardless)
TS-GW-006	Gateway PostAsync with duplicate idempotency_key	Returns existing posting result (no duplicate JE)

20. Development Phases

Phase 1 — Foundation (Week 1–2)

Task	Sub-task	Effort (days)
1A	Create JournalTypes entity, migration M1	0.5
1B	Seed 7 system journal types, migration M8	0.5
1C	Journal Types API (CRUD endpoints)	1
1D	Journal Types UI panel	1
1E	Alter FiscalPeriods status constraint, migration M2	0.5
1F	Create JournalEntries entity, migration M3	1
1G	Create JournalEntryLines entity, migration M4	1
1H	Create JournalEntrySequences entity, migration M5	0.5
1I	Create ReconcileGroups entity, migration M6	0.5
1J	ChartOfAccounts trigger for has_transactions, migration M7	0.5
1K	Register all new entities in FinanceDbContext	0.5
	Phase 1 Total	**7.5 days**

Phase 2 — Core Engine (Week 3–4)

Task	Sub-task	Effort (days)
2A	IJournalEntrySequenceService — sequence generation with row locking	1
2B	IJournalEntryService.CreateAsync — full validation logic	2
2C	IJournalEntryService.PostAsync — status transition + CoA balance update	1
2D	IJournalEntryService.CreateAndPostAsync — combined flow	0.5
2E	IJournalEntryService.UpdateDraftAsync / DeleteDraftAsync	1
2F	IJournalEntryService.GetByIdAsync / ListAsync with filters	1
2G	Journal Entry API endpoints (CRUD + post)	1.5
2H	Journal Entry list/detail UI panels	2
	Phase 2 Total	**10 days**

Phase 3 — Reversal & Reconciliation (Week 5-6)

Task	Sub-task	Effort (days)
3A	IJournalEntryService.ReverseAsync — mirror entry + auto-reconcile	2
3B	Reversal API endpoint	0.5
3C	IReconciliationService.ReconcileAsync — full/partial logic	2
3D	IReconciliationService.UnreconcileAsync	1
3E	IReconciliationService.GetOpenItemsAsync	0.5
3F	IReconciliationService.SuggestMatchesAsync	1
3G	Reconciliation API endpoints	1
3H	Reconciliation UI (open items list, match, confirm)	2
	Phase 3 Total	**10 days**

Phase 4 — Reports & Period Close (Week 7-8)

Task	Sub-task	Effort (days)
4A	ITrialBalanceService.GenerateAsync — full query + DTO mapping	1.5
4B	ITrialBalanceService.GetAccountBalanceAsync	0.5
4C	ITrialBalanceService.GetAccountLedgerAsync	1
4D	Trial Balance API endpoints	0.5
4E	Trial Balance UI panel with filters + export	2
4F	IPeriodCloseService — close/reopen/final-close period	1.5
4G	IPeriodCloseService.CloseYearAsync — full year-end logic	2
4H	IPeriodCloseService.PreviewYearEndCloseAsync	0.5
4I	Period Close API endpoints	0.5
4J	Period Close UI panel with year-end wizard	2
	Phase 4 Total	**12 days**

Phase 5 — Gateway Integration & Testing (Week 9–10)

Task	Sub-task	Effort (days)
5A	Enhanced JournalEntryBuilder — delegate to IJournalEntryService	1
5B	IJournalTypeResolver — DocumentType → JournalType mapping	0.5
5C	Integration test: Sale Invoice → Gateway → GL Engine → JE created	1
5D	Integration test: Purchase Invoice → Gateway → GL Engine → JE created	0.5
5E	Integration test: Parallel posting (internal GL + QB outbox)	1
5F	Integration test: Period closed → gateway returns error, QB outbox still written	0.5
5G	Unit tests for all business rules (BR-JE-001 through BR-TB-005)	3
5H	End-to-end test: Full accounting cycle (create → post → trial balance → period close → year-end)	2
5I	Performance test: Trial balance with 10K+ entries	0.5
5J	Security test: Permission checks on all endpoints	0.5
	Phase 5 Total	**10.5 days**

Summary

Phase	Description	Duration
Phase 1	Foundation (entities, migrations, journal types)	7.5 days
Phase 2	Core Engine (create, validate, post, list)	10 days
Phase 3	Reversal & Reconciliation	10 days
Phase 4	Reports & Period Close	12 days
Phase 5	Gateway Integration & Testing	10.5 days
Total		**50 days (10 weeks)**

---

21. Appendix A — Deferred Financial Addendum Roadmap

The following addendums build on the GL Engine established here:

Addendum	Title	Depends On	Key Deliverables
A48	AR/AP Sub-Ledgers	A47 (GL Engine)	Customer aging, vendor aging, payment allocation, dunning letters, customer/vendor statements
A49	Bank Reconciliation	A47 (GL Engine)	Bank feed import (OFX/CSV), match bank lines to JE lines, bank statement reconciliation, cleared balance tracking
A50	Multi-Currency Engine	A47 (GL Engine)	Exchange rate tables, realized gain/loss on payment, unrealized gain/loss revaluation, multi-currency trial balance
A51	Tax Configuration & Reporting	A47 (GL Engine)	Tax codes, tax rates by jurisdiction, tax-inclusive/exclusive pricing, VAT/GST/Sales Tax returns
A52	Financial Statements	A47 + A48 (GL + AR/AP)	Balance Sheet, Income Statement (P&L), Cash Flow Statement, configurable report layouts, comparative periods

Each addendum operates within the `finance` schema, uses `FinanceDbContext`, and posts through the Financial Posting Gateway (A44). The GL Engine's service interfaces (`IJournalEntryService`, `IReconciliationService`, `ITrialBalanceService`, `IPeriodCloseService`) serve as the foundation they build on.

End of Addendum 47